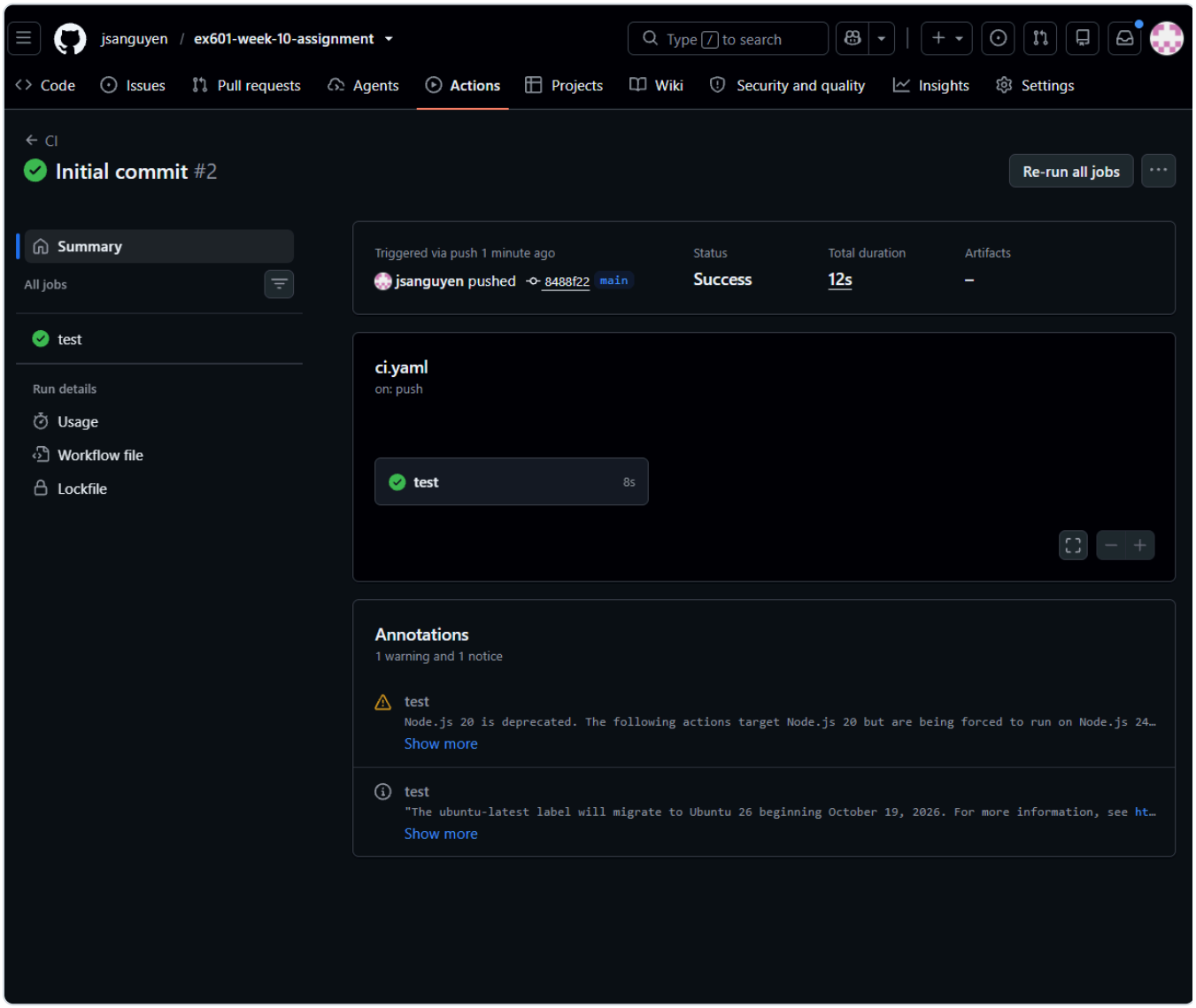
EX601 Week 10 Assignment — CI with GitHub Actions

Repository: https://github.com/jsanguyen/ex601-week-10-assignment

CI runs: https://github.com/jsanguyen/ex601-week-10-assignment/actions

Submission 1: Green run

Green run screenshot:



In one sentence: what event started this run, and who started it?

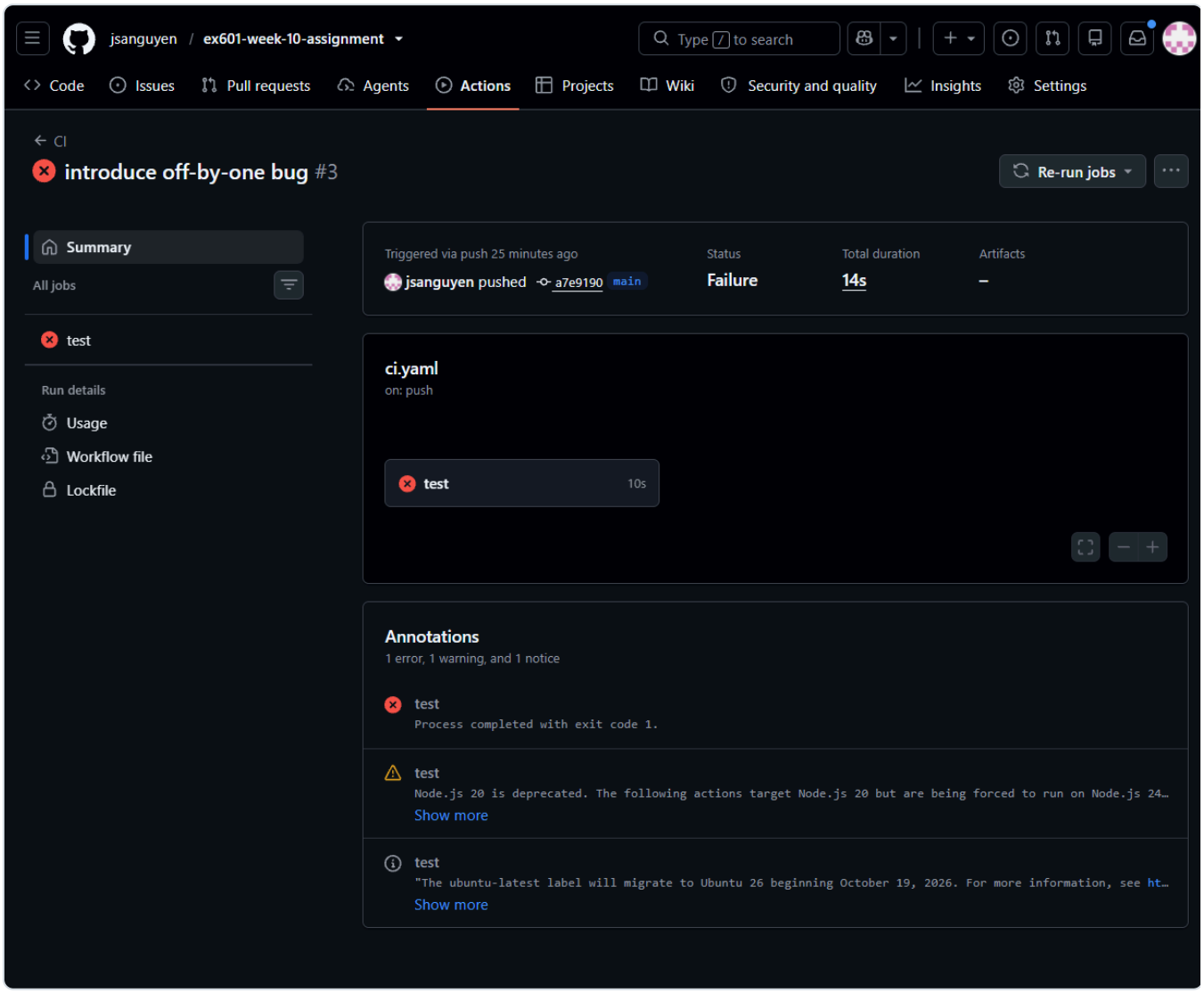
A push of a commit to `main` started this run and Jonathan Nguyen started it.

Submission 2: Red run, then green again

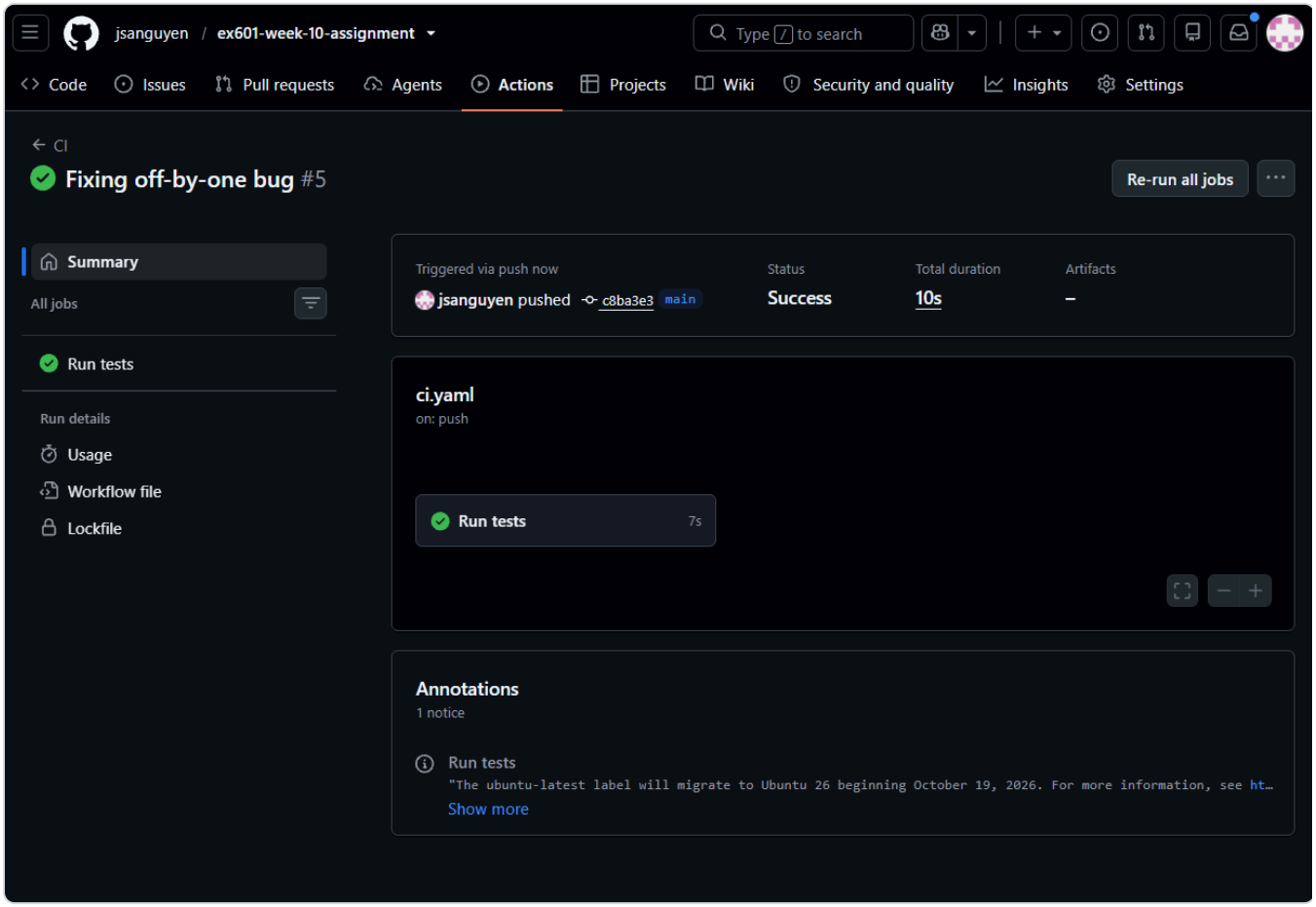
My prediction of which test would fail: `test_boundary_exactly_24_hours_is_free` — **was I right?** Yes

The bug changed `hours_before >= 24` to `hours_before > 24` in `fee_calculator.py`, so cancelling exactly 24 hours ahead was charged $25 instead of being free.

Red run screenshot:



Green-again screenshot:



In one sentence: what would this bug have cost if there were no pipeline?

Without a pipeline, the bug would have shipped unnoticed and charged every patient who cancelled exactly 24 hours ahead a $25 fee the clinic's own rule says they don't owe, costing refunds, complaints and patient trust.

What's in this repo

File	Purpose
`fee_calculator.py`	`cancellation_fee(hours_before)`: cancelling less than 24 hours before an appointment costs $25, while 24 hours or more ahead is free. Negative hours raise `ValueError`.
`test_fee_calculator.py`	pytest tests covering early cancellation, late cancellation, the exact 24-hour boundary, and negative input.
`requirements.txt`	Pinned test dependencies (pytest), shared by local runs and CI.
`.github/workflows/ci.yaml`	GitHub Actions workflow that runs the tests on every push.

Running the tests locally

```
pip install -r requirements.txt
pytest -v
```

How the CI pipeline works

Every push triggers the `CI` workflow. It can also be started by hand with the **Run workflow** button in the Actions tab. The workflow:

1. Checks out the repo on a fresh `ubuntu-latest` machine.
2. Installs Python 3.12, reusing cached pip downloads from earlier runs.
3. Installs the pinned dependencies from `requirements.txt`.
4. Runs `pytest -v`, which prints each test's name with PASSED or FAILED. If any test fails, this step fails and the whole run turns red.

The job has read-only access to the repo and is stopped after 10 minutes if it hangs.

AI Reflection

One improvement the AI suggested: It added more verbose logs. Tests run with pytest -v which will print each tests name with "PASSED" or "FAILED"

The gate in its version: ITs bascially exactly the same as the original. ci.yaml:28-29:

Would I merge its version? Yes, clearer logs, pinned deps and pip caching are all great, free additions that doesn't slow down the pipeline.

Reflection (100-150 words):

- Why is a pipeline that skips its tests worse than no pipeline?
  - With no pipeline, everyone knows nothing is gettinch checked which means that they will run the test themsevles, carefully review code with more attention, and assume that each merge contains a fair amount of risk which increases their vigilance. A pipeline with skipped tests will make people trust that the green pipeline check worked succesffully when in fact it is a red herring. Skipped tested pipelines will also really mess up your pipeline histroy because you'll see a wave of green pipelines in your history and it becomes tedious to figure out which, if any, are actually valid passes.
- What did the red run teach you that the green run could not?
  - The the tests are important and shows you what parts of the code could be wrong or incorrect. We use tests to catch bugs and edge cases. Having these built correctly protects our pipeline so that buggy code is not deployed since the pipeline literally cannot pass until those tests pass. But then the next thnig to consider is making those tests truly robust and accurate.

- One rule you will follow when a pipeline guards code you care about.
    - Never skip tests, linting or build steps. If they fail, let the pipeline fail. These exist to protect the quality of the code. But that also means the tests you built must be solid. Shoddy tests passing is just the same as skipping them,